

Module 2- Interpretability

Dr Suguna M

Module 2- Interpretability

- Difference between Interpretability and Explainability - Interpretability methods to explain Black-Box Model - Scope of Interpretability - Apply interpretability on Regression, Logistic regression, Generalized Additive Models, Decision Tree - Neural network interpretation -Evaluation of Interpretability

Interpretability

- “Interpretability is the degree to which a human can understand the cause of a decision.”(Biran, Or, and Courtenay V. Cotton. 2017. “Explanation and Justification in Machine Learning: A Survey.” In *Proceedings of the IJCAI-17 Workshop on Explainable Artificial Intelligence (XAI)*. https://www.cs.columbia.edu/~orb/papers/xai_survey_paper_2017.pdf)

Interpretability

- “A method is interpretable if a user can correctly and efficiently predict the method’s results”(Kim, Been, Rajiv Khanna, and Oluwasanmi Koyejo. 2016. “Examples Are Not Enough, Learn to Criticize! Criticism for Interpretability.” In Proceedings of the 30th International Conference on Neural Information Processing Systems, 2288–96. NIPS’16. Red Hook, NY, USA: Curran Associates Inc.)

Difference between Interpretability and Explainability

- Interpretability is about mapping an abstract concept from the models into an understandable form.
- Explainability is a stronger term requiring interpretability and additional context.
- Additionally, the term explanation is typically used for local methods, which are about “explaining” a prediction. Anyway, these terms are so fuzzy that the pragmatic approach is to see them as an umbrella term.
- **Interpretable machine learning - “extraction of relevant knowledge from a machine-learning model concerning relationships either contained in data or learned by the model” (Murdoch et al. 2019).**

Why XAI necessary?

- **Transparency:**
 - Helps users understand model behavior (especially in critical applications – healthcare, Finance, Law enforcement)
 - Ex: Cibil Score being reduced, Loan being rejected, Stock market investment
- **Trust:**
 - Builds confidence in AI systems (justification AI outputs)
- **Debugging:**
 - Identifies model errors or biases or unintended correlations.
 - Ex: Expert knowledge bias, Data bias, Feature selection bias
- **Compliance:**
 - Meets legal and ethical standards (GDPR and AI Act)

Fundamentals of XAI - Summary

- **Interpretability:** The degree to which a human can understand the cause of a decision. [The degree to which a human can consistently predict the model's result]
- **Transparency:** How clearly the AI system reveals its internal processes.
- **Post-hoc Explanations:** Explanations given after the model makes a decision.

XAI Techniques

- **LIME** (Local Interpretable Model-agnostic Explanations)
 - **SHAP** ((SHapley Additive exPlanations)
 - **Grad-CAM**
 - **DeepLIFT**
 - **Saliency Maps** – topographical representation of unique features
 - **Partial Dependence Plots (PDP)**
 - **Individual Conditional Expectation (ICE)**
 - **TCAV** (Testing with Concept Activation Vectors)
- To any ML model
- To any DNN model

Black Box Model

- A system that **does not reveal its internal mechanisms**.
- "Black box" describes models that cannot be understood by looking at their parameters. e.g. a neural network.
 - Model-agnostic methods for interpretability treat ML models as black boxes, even if they are not.
- The opposite of a black box is **White Box**
 - It is interpretable model
 - **Interpretable ML** are methods and models that are understandable to humans by the **behavior and predictions** of the systems.

Interpretability

Interpretability is the degree

- to which a human can understand the cause of a decision.
- to which a human can consistently predict the model's result

Understanding of artifacts given by system

- The explanation given by a model for its decisions
- The fidelity of explanation by the model – accurate explanation of the system

Understanding of a model can be measured in terms of interpretability of the model

- Interpretability is the metric to measure the understanding of model
- It measures the acceptability and usability of model.

The need for interpretability arises from an incompleteness in problem formalization

- Accuracy/Recall/ Precision/Confusion matrix for classification??
- WHAT $\Leftrightarrow$ Why
- Certain problems or tasks it is not enough to get the prediction (the what).
- The model must also explain how it came to the prediction (the why)

It is easy to identify cause-and-effect relationships for the interpretable machine learning system in I/P and O/P of system.

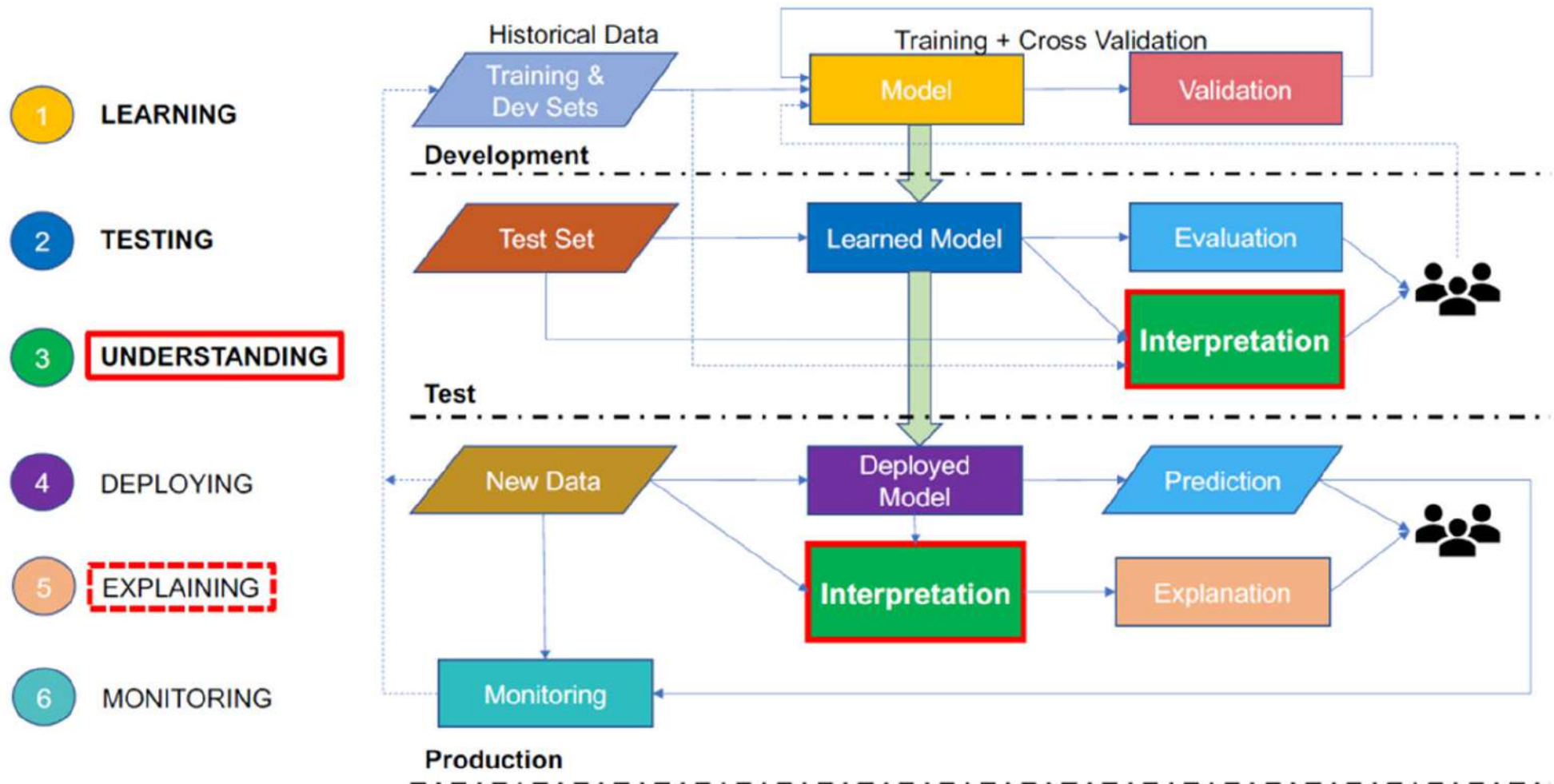
- A specific object is part of an image (output) could be certain dominant patterns in the image (input). – Reason for the output by the system.

Interpretability Vs Explainability

Interpretability	Explainability
How does the model work?	Why did the model make this decision?
Transparent models	Black-box models
Intrinsic (built-in)	Post-hoc (after training)
Understand structure	Understand behavior

- The interpretability alone is **insufficient** and that the presence of **explainability** is also of fundamental importance.
- Interpretability to be a broader term than explainability.
 - **Explainability** – developers
 - **Interpretability** – Users/Policy makers/ Variety of users.

Building an explanation producing system to generate a human-readable explanation for the end users of your system.



Scope of Interpretability / transparency

- *How does the **algorithm** create the model?*
 - Algorithm Transparency - learns a model from the data and the relationships in it
- *How does the trained model make **predictions**?*
 - Global, Holistic Model Interpretability - Knowledge of algorithm, the model, data, predication (distribution of your target outcome based on the features)
- *How do **parts** of the model **affect predictions**?*
 - Global Model Interpretability on a *Modular Level*
- *Why did the model make a **certain prediction for an instance or specific predictions for a group of instances**? ?*
 - Local Interpretability for a Single Prediction or Group of Predictions.

Algorithm Transparency

- The understanding of how
 - the algorithm learns a model from the data
 - The relationships it can learn.
 - CNN to classify images,
 - The algorithm learns edge detectors and filters on the lowest layers.
 - The specific model that is learned in the end is different than lowest layers and used for prediction.
- Algorithm transparency only requires **knowledge of the algorithm** and not of the data or learned model.
- Algorithms such as the **least squares method** for linear models gives **high transparency**.
- Deep learning approaches are not understood well and **less transparent**. The inner workings of DL are the focus of ongoing research.
 - DL - **pushing a gradient through a network with millions of weights**

Global, Holistic Model Interpretability

- To understand the entire model, one needs following
 - Knowledge of algorithm, The model, data, predications.
 - to understand the distribution of your target outcome based on the features.
 - It is difficult to develop such an understanding.
- To understand how the model makes decisions, based on
 - a holistic view of its features
 - Which features are important and what kind of interactions between them take place?
 - each of the learned components
 - such as weights, other parameters, and structures.
- Human have short-memory and can develop global model interpretability for few features (3 features) and weights.
 - Can not develop understanding of 4 or more features and 4 dimensional space.

Evaluation of Interpretability

Assesses how effectively a model or an explanation method helps users understand:

- **How the model works**
- **Why a particular prediction was made**
- **What factors influenced the decision**

Doshi-Velez and Kim (2017) propose three main levels for the evaluation of interpretability

- **Application level evaluation** (real task)
- **Human level evaluation** (simple task)
- **Function level evaluation** (proxy task)

Interpretability

```
graph TD; A[Interpretability] --> B[Build interpretable ML models]; A --> C[Derive explanations for complex ML models]; B --- D[Model-based interpretability]; C --- E[Post-hoc interpretability]; E --> F[Black box]; E --> G[White box]; F --- H["Model parameters unknown. Derive explanations using the correlation b/w inputs and outputs."]; G --- I["Model parameters are known"];
```

Build interpretable
ML models

Model-based
interpretability

Derive explanations
for complex ML models

Post-hoc
interpretability

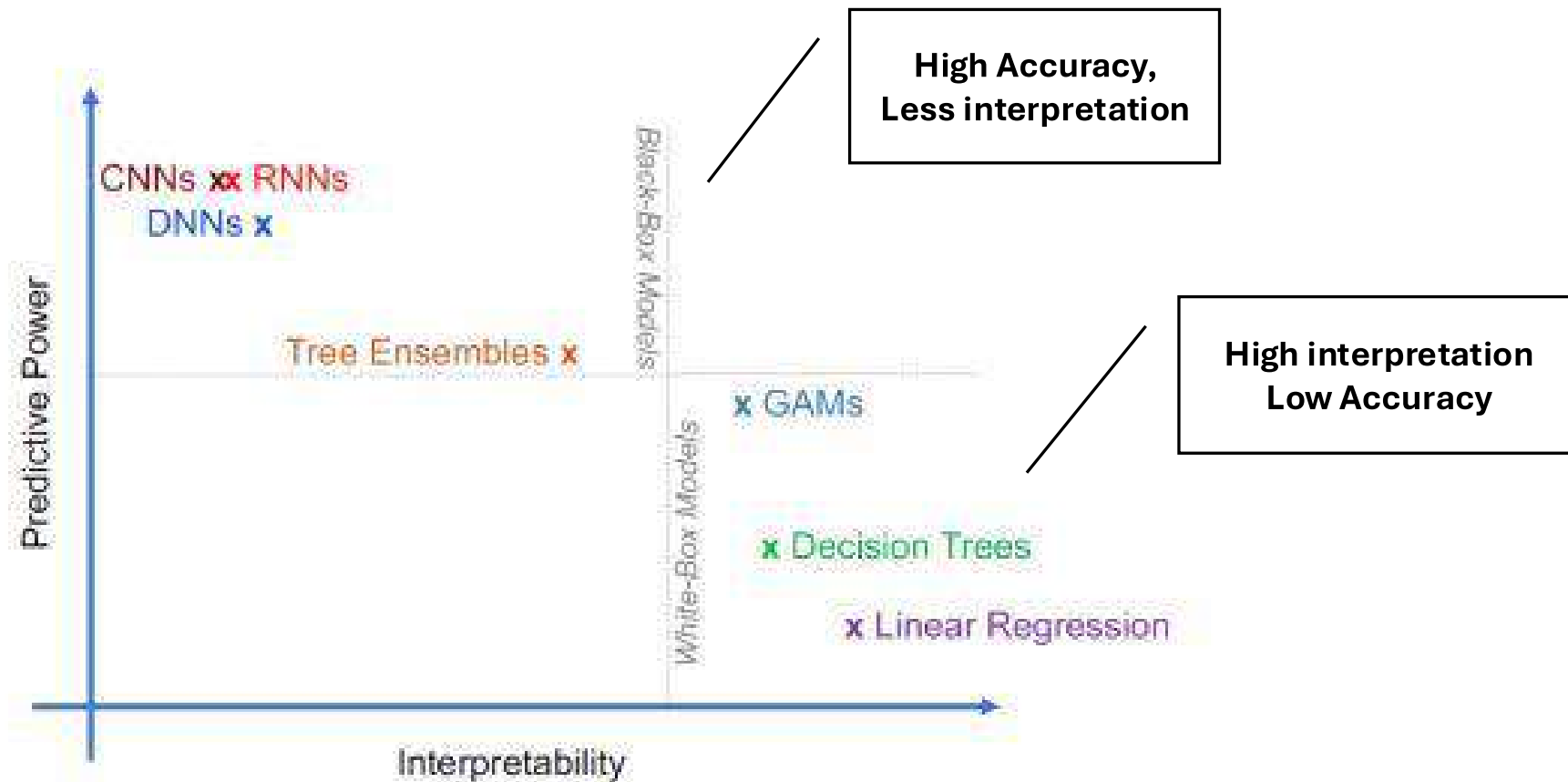
Black box

Model parameters unknown.
Derive explanations
using the correlation b/w
inputs and outputs.

White box

Model parameters are known

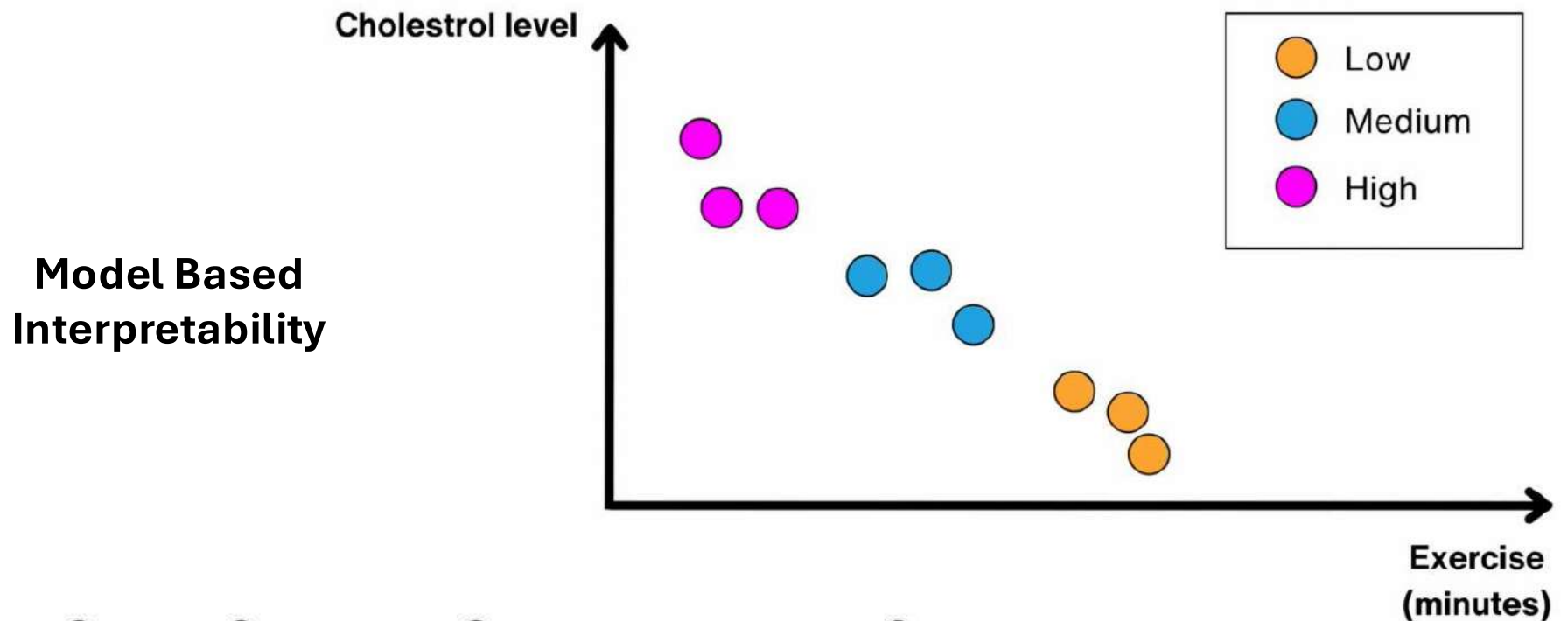
Black box vs. White box models



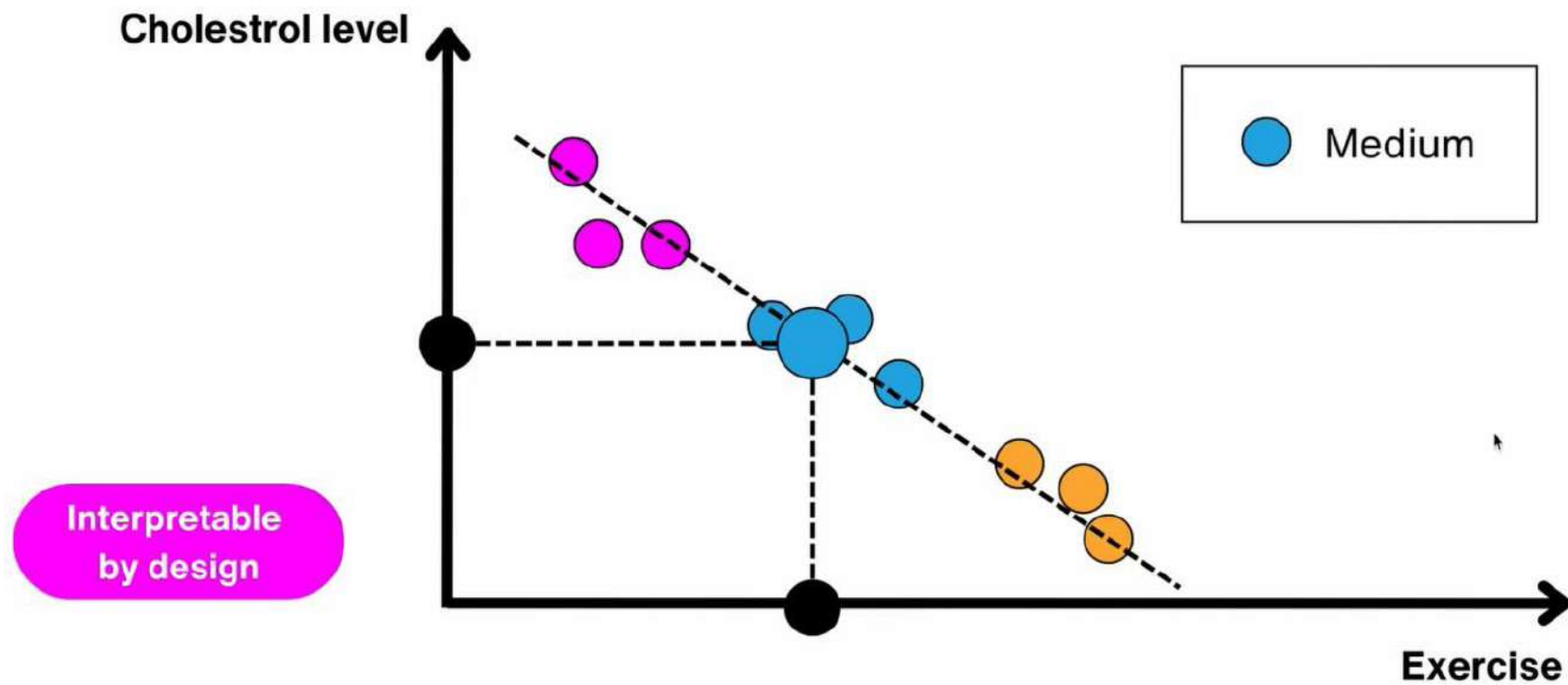
White box model – Interpretable model

White-Box Model	Machine Learning Task(s)
Linear Regression	Regression
Logistic Regression	Classification
Decision Trees	Regression and Classification
Generalized Additive Models (GAMs)	Regression and Classification

Interpretation on Regression



$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$



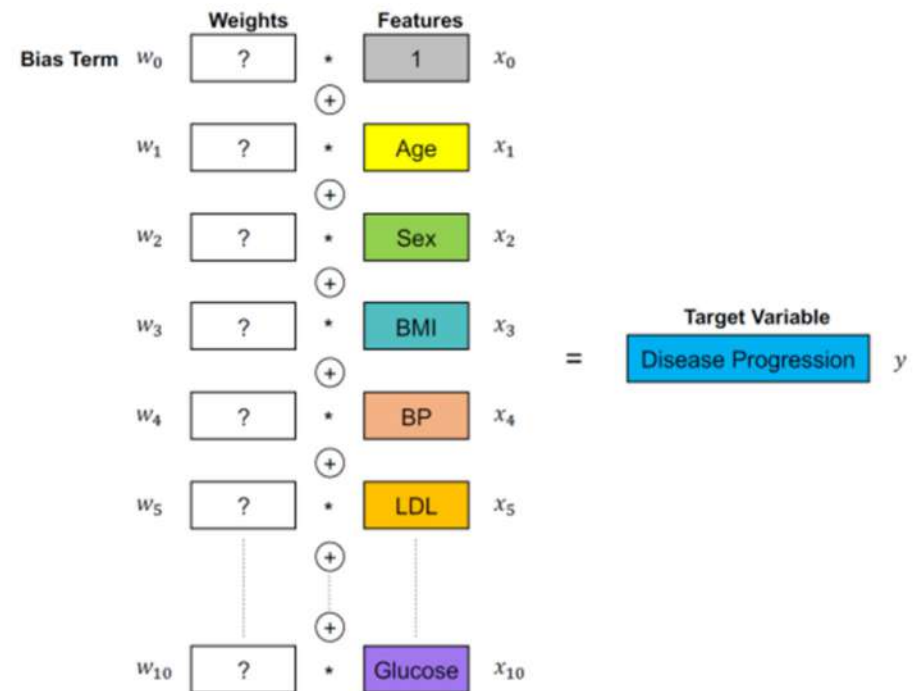
Regression Model itself is self interpretable - it is clear why the data was classified as low/ medium / high

Interpretability for Regression

- Diabetes detection
 - Age, gender , BMI, BP
 - LDL (the bad cholesterol)
 - HDL (the good cholesterol)
 - Total Cholesterol
 - Thyroid Stimulating Hormone
 - Low Tension Glaucoma
 - Fasting Blood Glucose
- 10 i/p , 1 o/p
- Real and continuous value

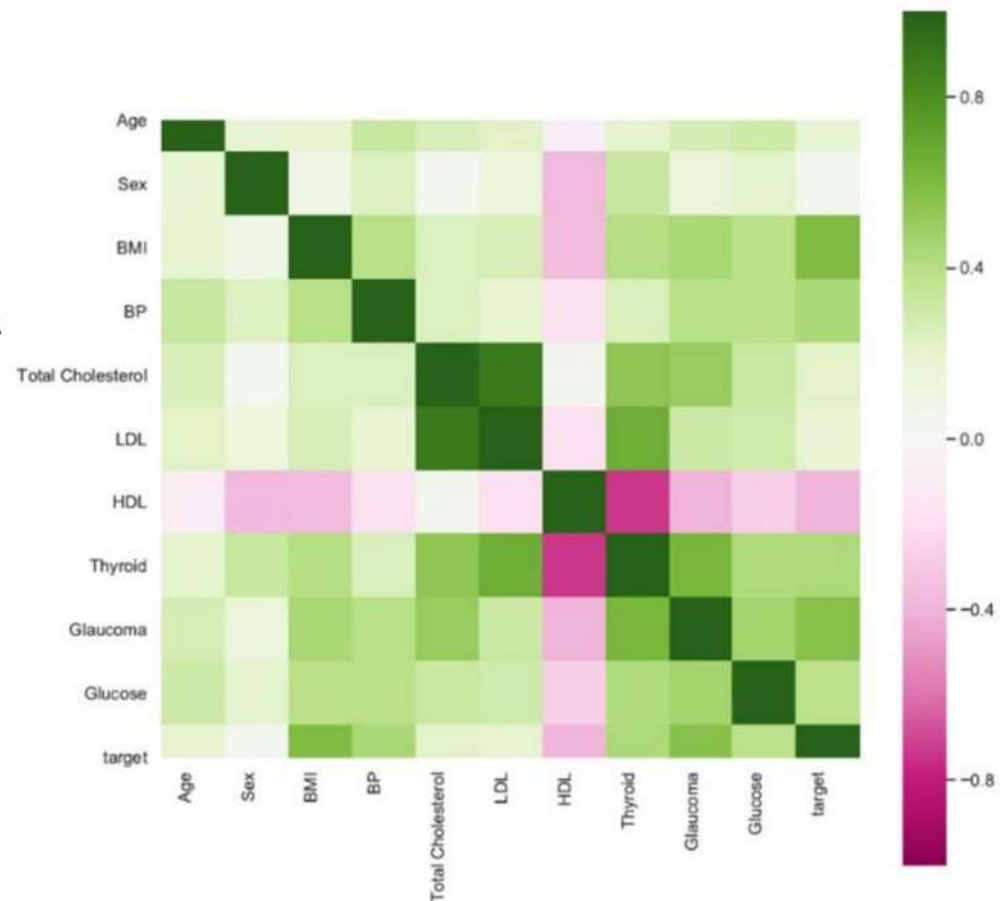
$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

$$= w_0 + \sum_{i=1}^n w_i x_i$$



- Correlation of i/p with i/p
 - Multicollinearity
 - Interaction between total cholesterol & LDL, Glaucoma

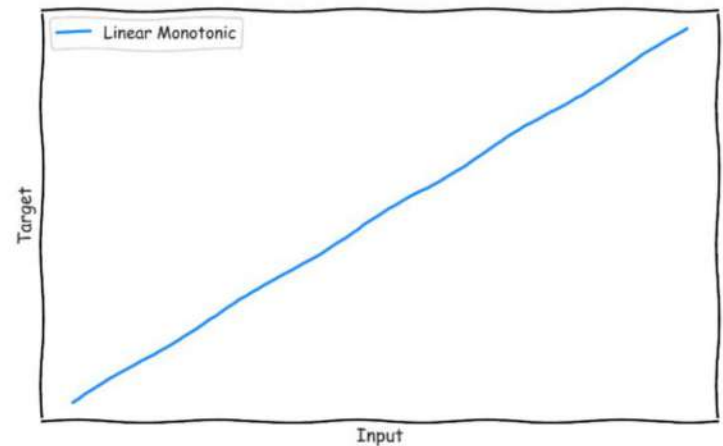
High => 0.7 and above,
Moderately high 0.7 – 0.5
Low 0.5 – 0.3
No correlation => less 0.3
- Correlation of i/p with o/p
 - Target & BMI, BP, Thyroid, Glaucoma, glucose



An input feature is highly correlated with one or more other features

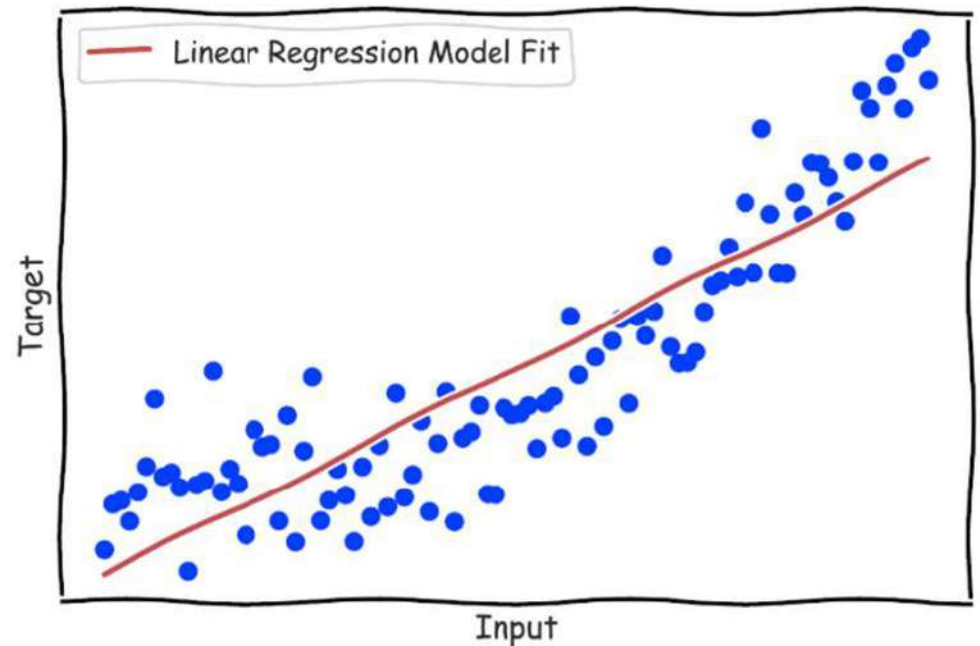
Interpreting Regression

- 10 features => 10 dimensional space
- White box Interpretable model => a weighted sum of the input features.
 - Positive weight
 - Negative weight
 - Linear – monotonic o/p
- Importance of a feature – Value of weight



Regression

- Relation of input to output is not linear
- If correlation are overlooked
 - Interpreting the model will help expose such issues.
 - Better way of interpreting the regression model.



Interpretation of a weight

Interpretation of a weight in the linear regression model depends on the type of the corresponding feature

- Numerical feature
- Binary feature
- Categorical feature with multiple categories
- Intercept β_0

$$R^2 = 1 - SSE/SST$$

- An increase of feature by one unit increases the prediction for y by units when all other feature values remain fixed.
- A feature that takes one of two possible values for each instance
- A feature with a fixed number of possible values
- The intercept is the feature weight for the “constant feature”, which is always 1 for all instances

Visual Interpretation

- **Weight Plot** - information of the weight table (weight and variance estimates) can be visualized
- **Effect Plot** - more meaningful when weights are multiplied by the actual feature values. weight per feature times the feature value of an instance: $\text{effect}_j^{(i)} = w_j x_j^{(i)}$
 - Feature effect plot shows the distribution of effects (= feature value times feature weight) across the data per feature.

Encoding of Categorical Features

- Treatment coding
- Effect coding
- Dummy coding

Explain Individual Predictions

- How much has each feature of an instance contributed to the prediction?
- The effect plot for one instance shows the effect distribution and highlights the effects of the instance of interest.
- Compare the individual effects with the distribution of effects in the data

Advantage

- Weighted sum makes it transparent how predictions are produced

Disadvantages

- Linear regression models can only represent linear relationships
- Interpretation of a weight can be unintuitive because it depends on all other features
- Linear models are also often not that good regarding predictive performance

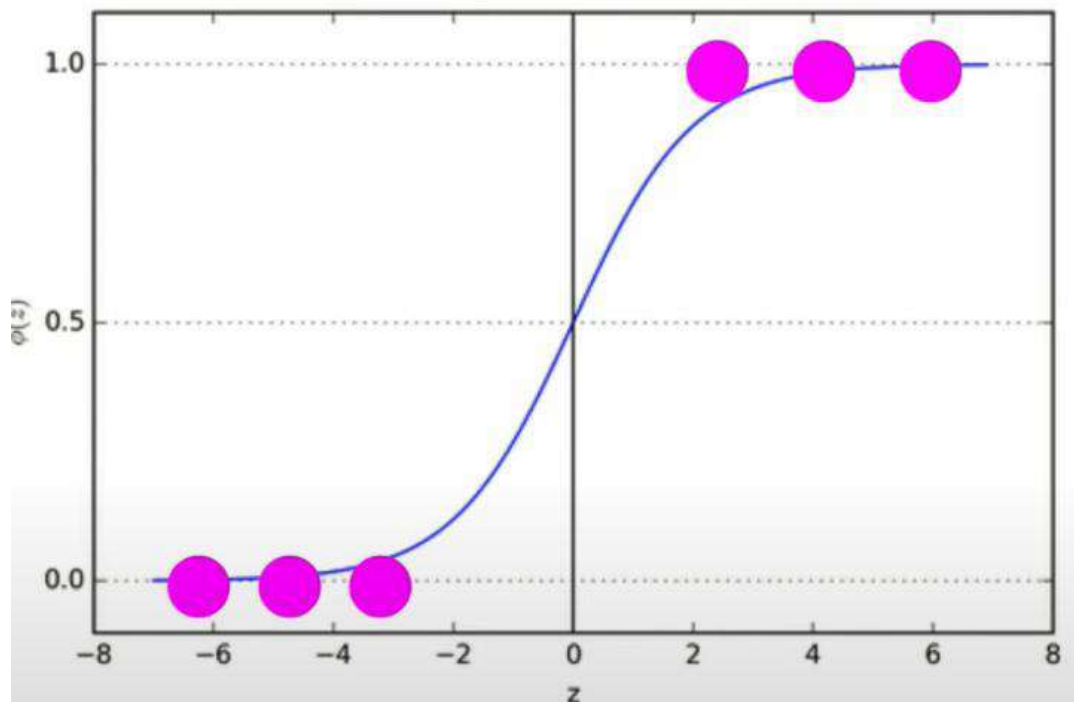
Logistic Regression

- Extension of Regression
- Output is binary (0 or 1)

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

Interpretation of Logistic Regression



- Model inherent
- It is clearly known why the data point is classified as 0 or 1
- Variables (z, β, x, fn) are sensitive to each other

Interpretation – Logistic Regression

- Interpretation of the weights in logistic regression differs from linear regression
- Weights do not influence the probability linearly (since outcome is 0 or 1)
- Need to reformulate the equation

Interpretations for the logistic regression model with different feature types

- Numerical feature
- Binary categorical feature
- Categorical feature with more than two categories
- Intercept β_0

Any change in feature value or category – lead to change in $\exp(\beta_i x_i)$

Disadvantage

- Interpretation is more difficult because the interpretation of the weights is multiplicative and not additive.
- Suffer from complete separation

Challenges in Interpretability

- **Multicollinearity:** Affects coefficient reliability.
- **Feature scaling:** Can mislead interpretation.
- **Categorical encoding:** May dilute effect per category.
- **Overfitting:** May look interpretable but performs poorly.

Interpretation properties

Explainable Properties	Regression (linear)	Logistic Regression
Local / Global	Global	Global
Linearity	Linear	Linear
Monotonic/ Non-Monotonic	Monotonic	Monotonic
Feature interaction captured	No	No
Model Complexity (suited)	Low	Medium

Interpretation on Regression - Summary

- Both regression and logistic regression are inherently interpretable due to their **linear structure**.
- Coefficients give direct insights into feature impact.
- Tools like **SHAP**, **LIME**, **PDP** enhance understanding, especially with complex relationships or transformed features.

LIME - To understand individual predictions

SHAP - Considers feature interactions, assigns fair contributions

PDP - Visualize effect of a feature while keeping others constant

ICE - Similar to PDP but for individual data points

Reference

- Molnar, Christoph. “Interpretable machine learning. A Guide for Making Black Box Models Explainable”, 2019.
<https://christophm.github.io/interpretable-ml-book/>.
- Explainable Artificial Intelligence: An Introduction to Interpretable Machine Learning, Uday Kamath: John Liu, Springer, ISBN 9783030833558

Causal inference and interpretable ML

- **Causal inference** is a subject from statistics which is related, but distinct, from interpretable machine learning. **Causal inference methods focus solely on extracting causal relationships from data, i.e., statements that altering one variable will cause a change in another.** In contrast, interpretable ML, and most other statistical techniques, is used to describe general relationships. Whether or not these relationships are causal cannot be verified through interpretable ML techniques, as they are not designed to distinguish between causal and noncausal effects.

Interpretability methods to explain Black-Box Model

- The most effective modeling approaches today are “black boxes” — models with mathematical behavior too complicated to directly interpret the effect that individual input features have on the model’s output. Fortunately, several interpretability methods exist which can be layered on top of any arbitrary black box model to approximate how:
 - Individual features impact predictions globally across the model (global interpretability)
 - Individual predictions are locally influenced by feature values (local interpretability)

Local Model-Agnostic Methods

- Ceteris Paribus Plots
- Individual Conditional Expectation (ICE)
- LIME
- Counterfactual Explanations
- Scoped Rules (Anchors)
- Shapley Values
- SHAP

Global Model-Agnostic Methods

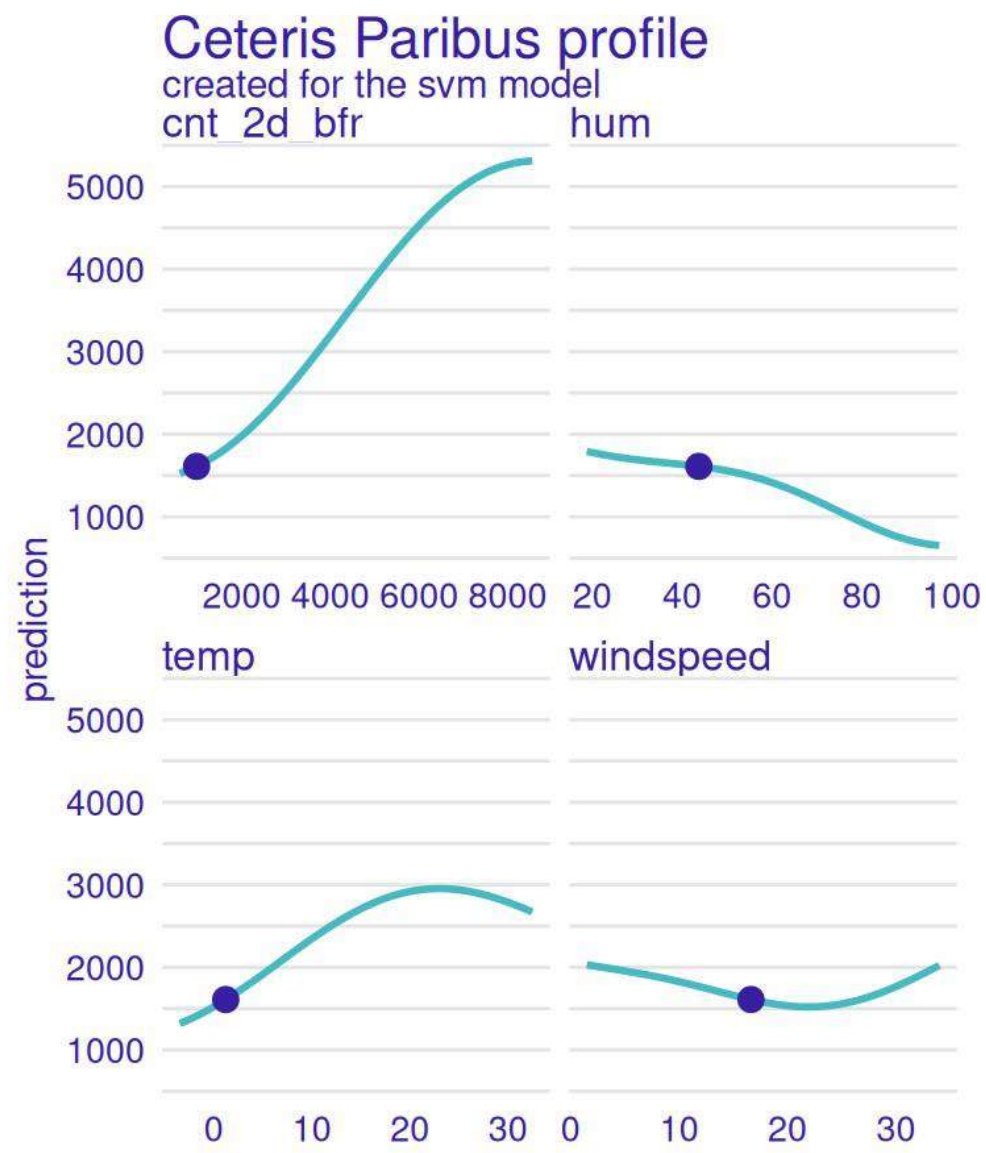
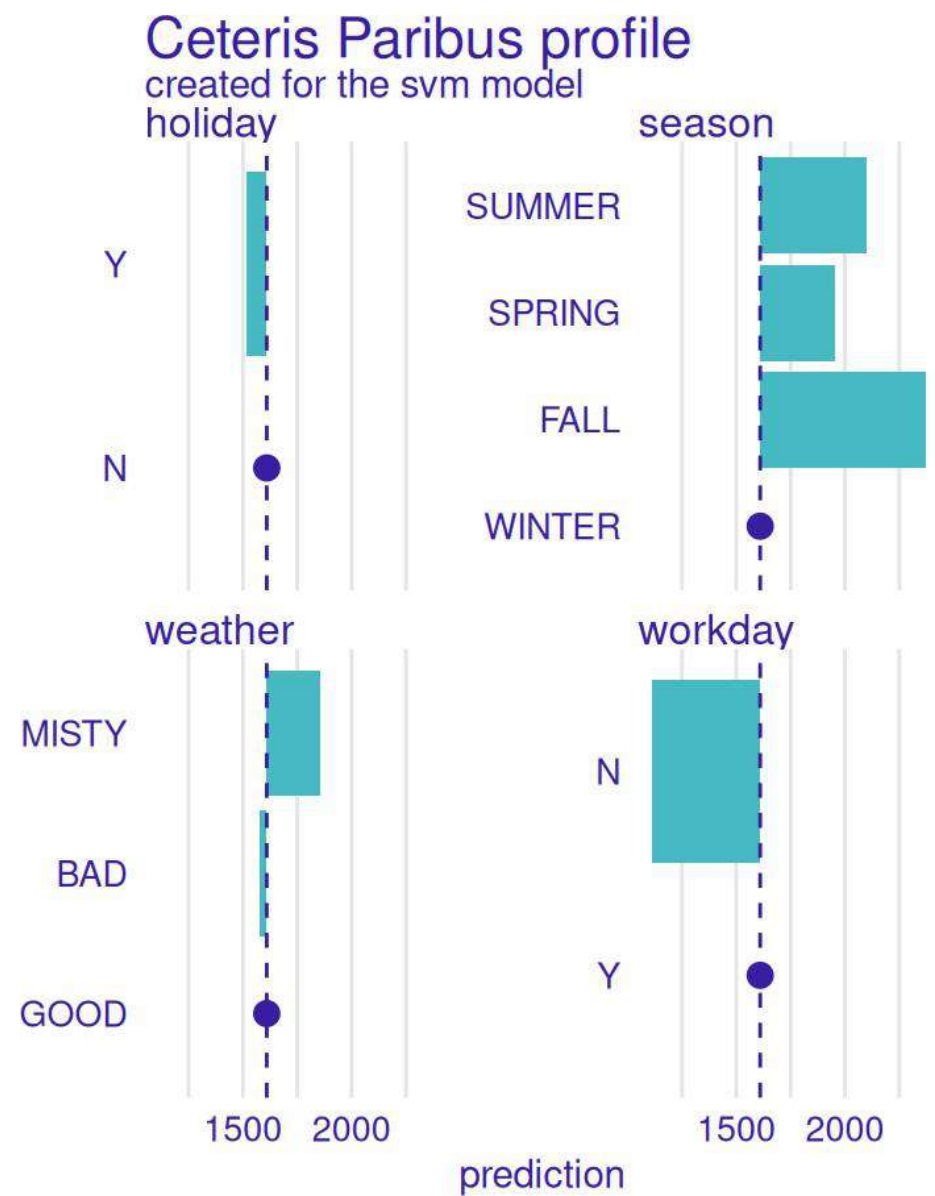
Global Model-Agnostic Methods

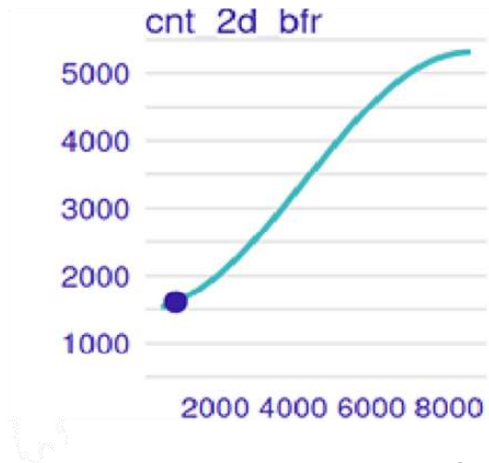
- Partial Dependence Plot (PDP)
- Accumulated Local Effects (ALE)
- Feature Interaction
- Functional Decomposition
- Permutation Feature Importance
- Leave One Feature Out (LOFO) Importance
- Surrogate Models
- Prototypes and Criticisms

Ceteris Paribus Plots

- **Ceteris paribus plots**, in the context of machine learning, are visualizations that show how a model's prediction changes when one input feature is varied while all other features are held constant (ceteris paribus, meaning "all other things being equal").
- These plots help in understanding the influence of individual features on the model's output and can be used for model explanation and comparison

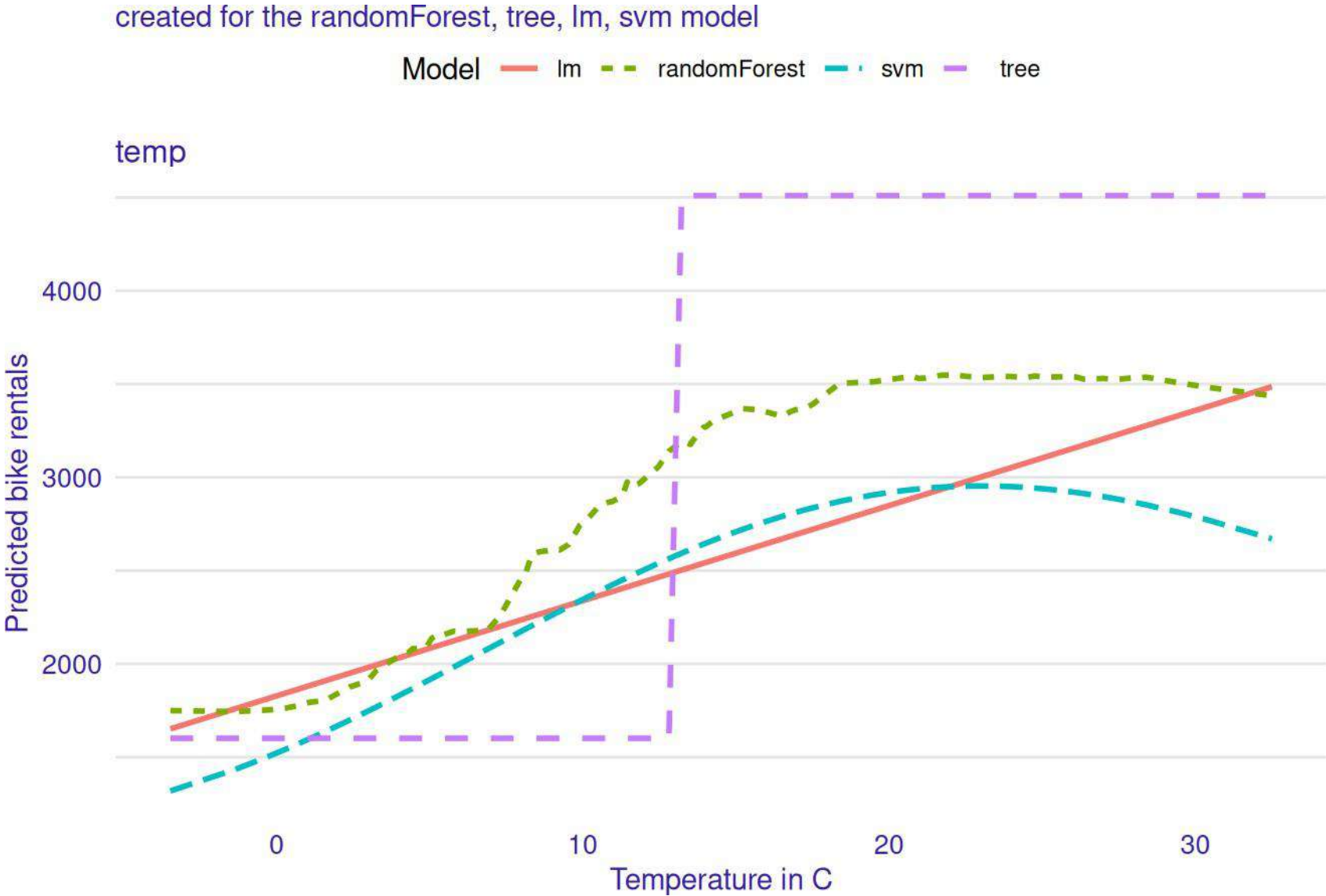
How changing individual features changes the predicted number of bike rentals.





- **X-axis:** Number of bikes rented 2 days ago
- **Y-axis:** Predicted bike rentals for today.
- The line rises sharply: if there were more bikes rented 2 days ago, the model predicts **more rentals today**.
- Interpretation: This feature has **the strongest influence** — the model is capturing momentum.

Ceteris paribus curves for the bike prediction task for different models: linear model, random forest, SVM, and decision tree.



- `pyCeterisParibus` is a Python library based on an R package `CeterisParibus`. It implements Ceteris Paribus Plots.
- They allow understanding how the model response would change if a selected variable is changed. It's a perfect tool for What-If scenarios. Ceteris Paribus is a Latin phrase meaning all else unchanged. These plots present the change in model response as the values of one feature change with all others being fixed. Ceteris Paribus method is model-agnostic - it works for any Machine Learning model. The idea is an extension of PDP (Partial Dependency Plots) and ICE (Individual Conditional Expectations) plots. It allows explaining single observations for multiple variables at the same time.

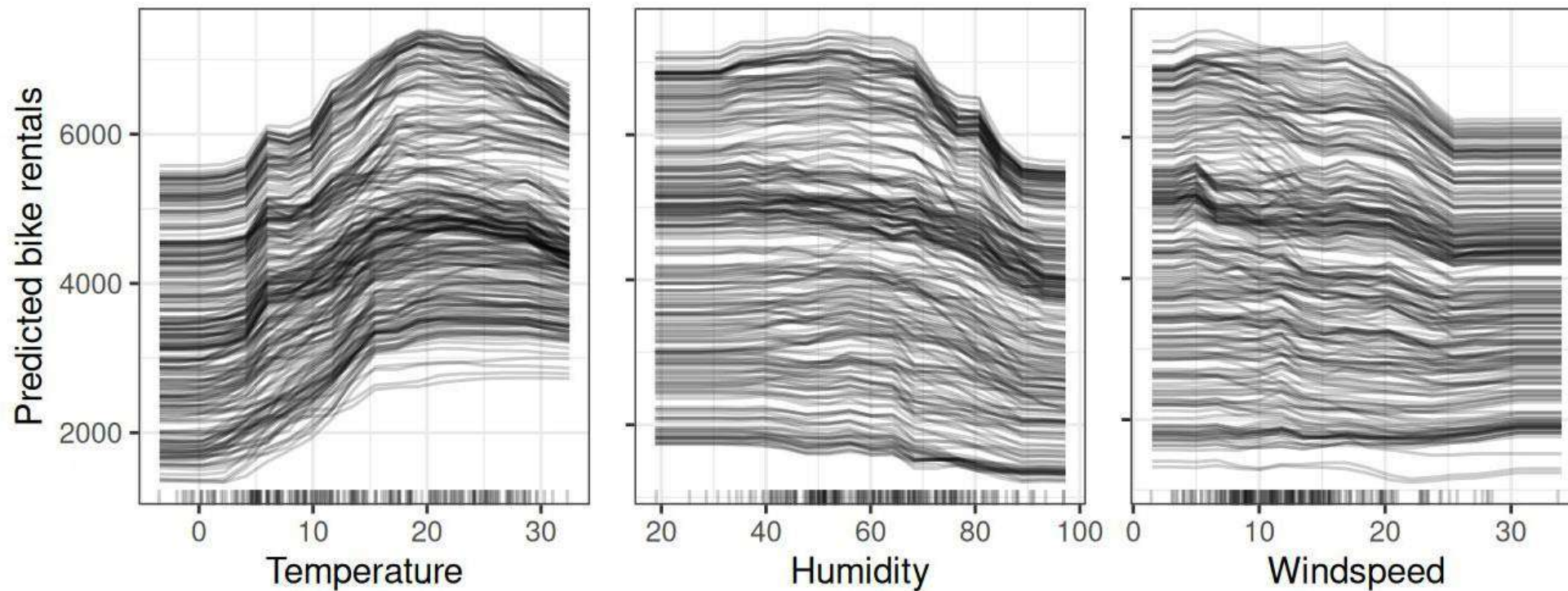
Individual Conditional Expectation (ICE)

An Individual Conditional Expectation (ICE) plot is a visualization technique used to understand and interpret complex machine learning models. It displays the relationship between a specific feature and the model's predictions for individual data points. By examining these plots, practitioners can gain insights into how a model relies on specific features, identify issues with model predictions, and guide feature selection for model training.

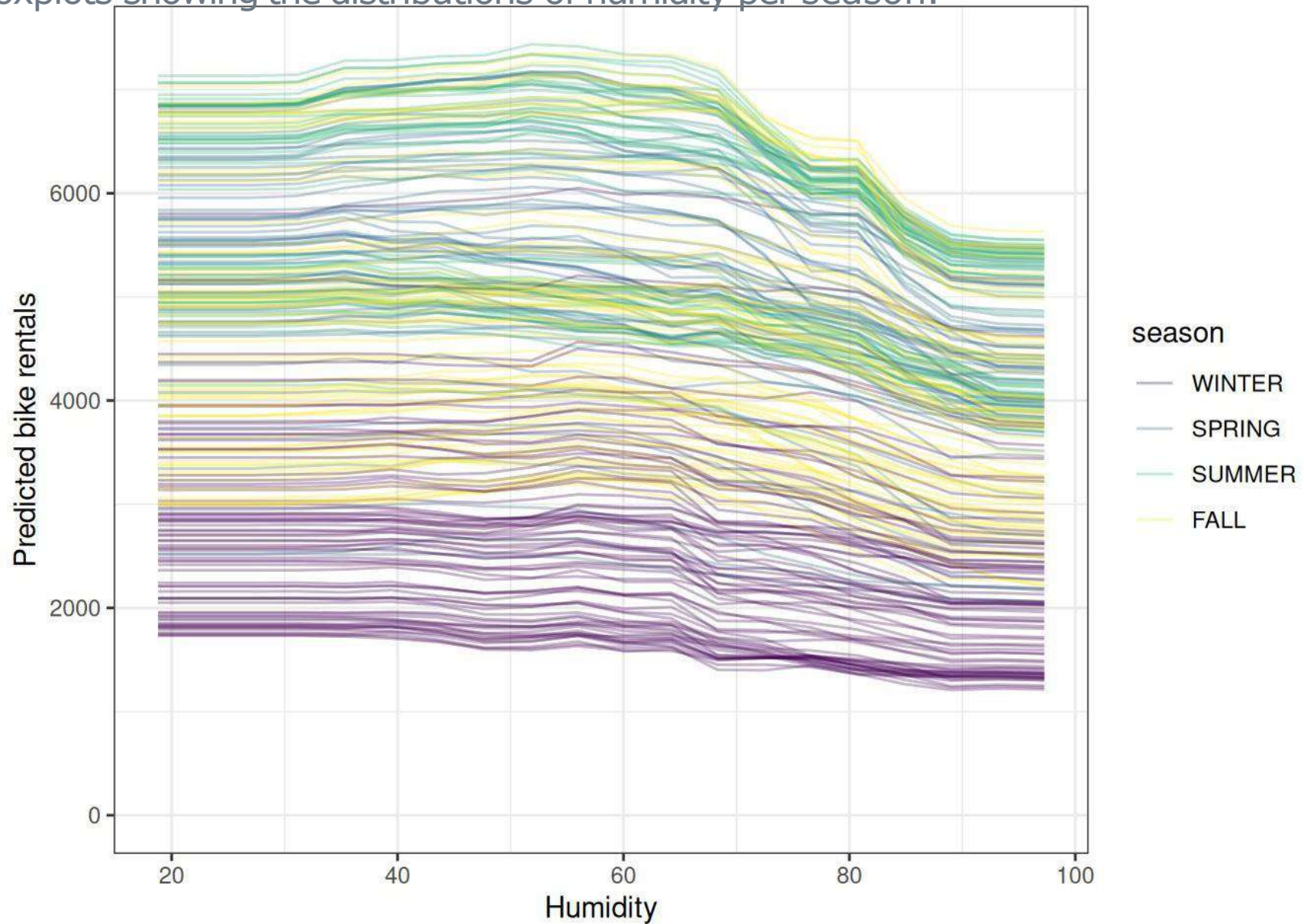
ICE plots for the bike rental prediction.

The underlying prediction model is a random forest.

All curves seem to follow the same course, so there are no obvious interactions.



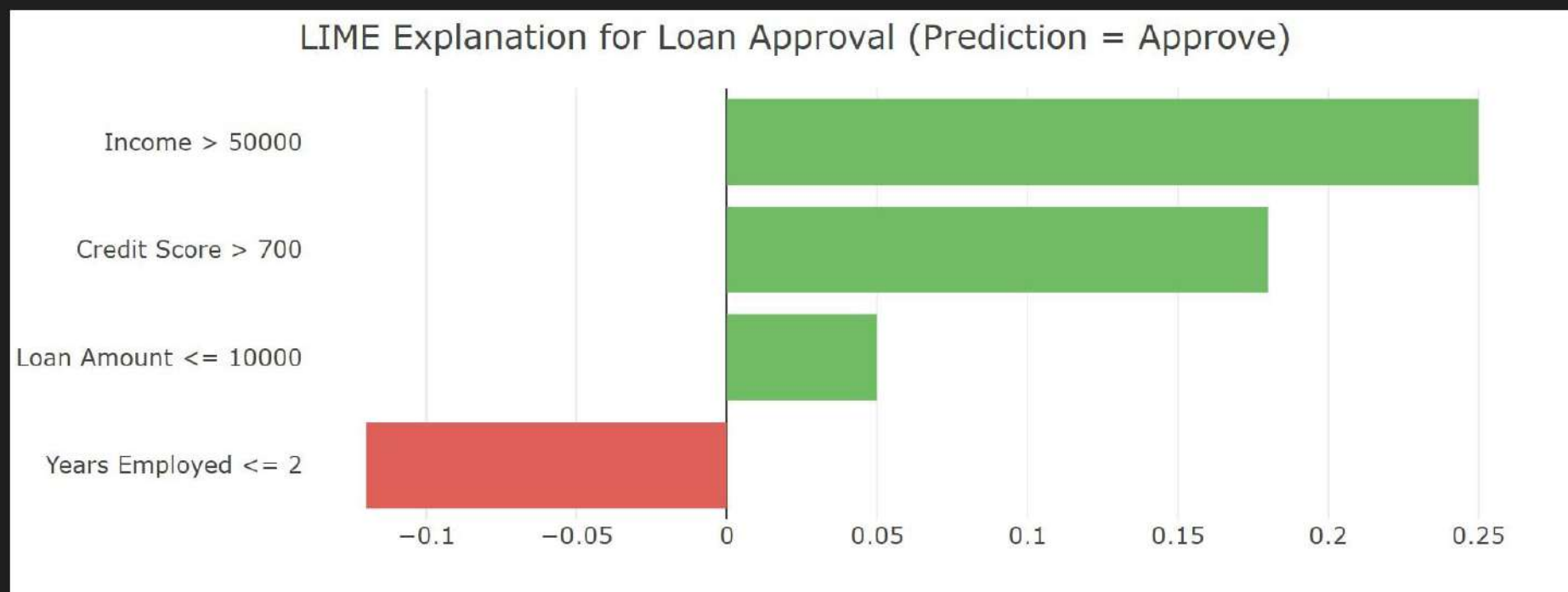
CE curves for the random forest predicting bike rentals. Lines are colored by the season.
ICE plots are boxplots showing the distributions of humidity per season.



Local interpretable model-agnostic explanations (LIME)

- Local surrogate models are interpretable models that are used to explain individual predictions of black box machine learning models. Local interpretable model-agnostic explanations (LIME), proposed by Ribeiro, Singh, and Guestrin (2016), is an approach for fitting surrogate models. Surrogate models are trained to approximate the predictions of the underlying black box model.

Visualizations often use bar charts to display these contributions:



Feature contributions for a single loan approval prediction. Green bars indicate features supporting the 'Approve' outcome, while red bars indicate features opposing it. The length of the bar shows the magnitude of the contribution.

Model to classify a ball either as a football or a basketball.

It's a basketball!



It's a football!



It's a basketball!



It's a football!



It's a basketball!



It's a football!



Interpreting model decision using LIME

It's a basketball!



It's a football!



It's a basketball!



It's a football!



It's a basketball!



It's a football!



- It turns out that our classifier was correctly predicting a ball as a football because of the human body parts, not because of the ball itself.
- So our classifier wasn't trying to classify between a football and a basketball, but to classify between a human body parts and a basketball.
- Obviously, it's not what we want and thus, we shouldn't trust our model only based on its accuracy.

Counterfactual explanation

- A counterfactual explanation describes a causal situation in the form:
“If X had not occurred, Y would not have occurred.”
- For example: “If I hadn’t taken a sip of this hot coffee, I wouldn’t have burned my tongue.” Event Y is that I burned my tongue; cause X is that I had a hot coffee.
- Thinking in counterfactuals requires imagining a hypothetical reality that contradicts the observed facts (for example, a world in which I’ve not drunk the hot coffee), hence the name “counterfactual.”
- The ability to think in counterfactuals makes us humans so smart compared to other animals.

- import dice_ml
- **DiCE: Diverse Counterfactual Explanations for Machine Learning Classifiers**
- e1 = exp.generate_counterfactuals(x_test[0:1], total_CFs=2, desired_class="opposite")
e1.visualize_as_dataframe(show_only_changes=False)

	age	workclass	education	marital_status	occupation	race	gender	hours_per_week	income
0	29	Private	HS-grad	Married	Blue-Collar	White	Female	38	0

Diverse Counterfactual set (new outcome: 1)

	age	workclass	education	marital_status	occupation	race	gender	hours_per_week	income
0	29	Government	HS-grad	Single	Blue-Collar	White	Female	38	1
1	29	Government	Assoc	Married	Blue-Collar	White	Female	38	1

Scoped Rules (Anchors)

- The anchors method explains individual predictions of any black box classification model by finding a decision rule that “anchors” the prediction sufficiently.
- A rule anchors a prediction if changes in other feature values do not affect the prediction.
- Anchors utilizes reinforcement learning techniques in combination with a graph search algorithm to reduce the number of model calls (and hence the required runtime) to a minimum while still being able to recover from local optima.

Checking Wine Quality

```
IF 'sulphates' > 0.73 AND 'volatile acidity' <= 0.39 AND 'alcohol' > 10.20 AND 'citric acid' <= 0.26
AND 'density' <= 1.00 AND 'pH' <= 3.31 AND 'fixed acidity' > 9.20 AND 'free sulfur dioxide' <= 21.00
AND 'chlorides' <= 0.08
THEN PREDICT 'quality' = 7
WITH Precision = 0.719 AND Coverage = 0.005
IF 'alcohol' > 10.20 AND 'residual sugar' <= 2.20 AND 'density' > 1.00 AND 'volatile acidity' <= 0.39
THEN PREDICT 'quality' = 6
WITH Precision = 0.762 AND Coverage = 0.006
IF 'alcohol' > 11.10 AND 'sulphates' <= 0.73 AND 'volatile acidity' <= 0.52
THEN PREDICT 'quality' = 6
WITH Precision = 0.739 AND Coverage = 0.1014
IF 'alcohol' <= 10.20 AND 'total sulfur dioxide' > 62.00
THEN PREDICT 'quality' = 5
WITH Precision = 0.757 AND Coverage = 0.181
IF 'sulphates' <= 0.55 AND 'alcohol' <= 11.10 AND 'volatile acidity' > 0.39
THEN PREDICT 'quality' = 5
WITH Precision = 0.705 AND Coverage = 0.192
```

- Anchor Explanation:
- Anchor: ['MedInc $\leq$ 2.57', 'AveOccup $>$ 3.28']
- Precision: 1.0
- Coverage: 0.0936
- If Median Income (MedInc) $\leq$ 2.57 AND Average Occupancy (AveOccup) $>$ 3.28 Then the model will predict the same label as it did for this instance (e.g., low house value).

- Anchors are high-precision rules that "anchor" a prediction by showing which features, when fixed, keep the prediction stable.
- Example for a loan approval model:
- “If income > ₹50,000 and credit score > 700, then the model always predicts ‘Approved’, regardless of other features.”
- `pip install alibi`
- `from alibi.explainers import AnchorTabular`


```
pip install anchor-exp  
from anchor import utils  
from anchor import anchor_tabular
```

```
IF 'alcohol' > 11.10 AND 'total sulfur dioxide' > 22.00  
THEN PREDICT 'quality' = 6  
WITH Precision = 0.7201 AND Coverage = 0.1432
```

Shapley values

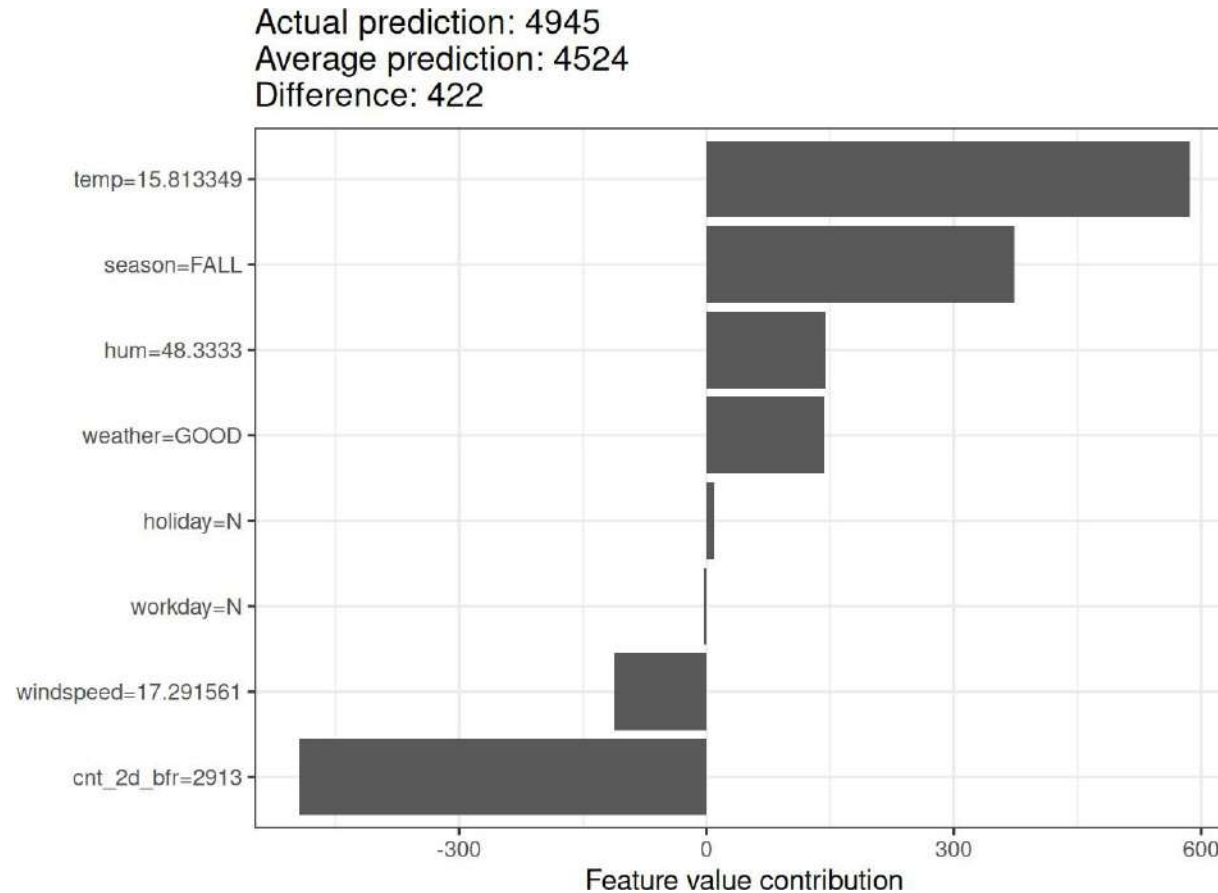
- A prediction can be explained by assuming that each feature value of the instance is a “player” in a game where the prediction is the payout.
- Shapley values – a method from coalitional game theory tell us how to fairly distribute the “payout” among the features.

Import shap for shapley values

import shap # `pip install shap` if needed

We use the `shap_values` method from the SHAP library to get Shapley values.

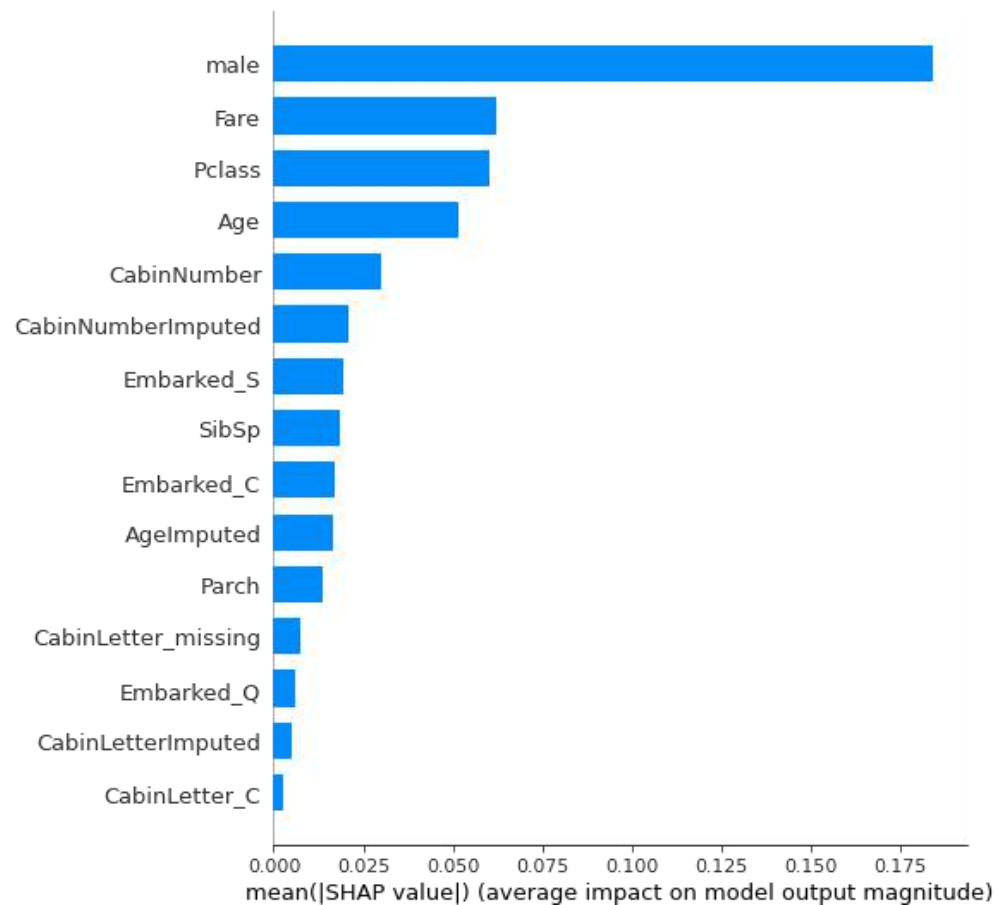
We use the `explainer` method from the SHAP library to get Shapley values along with other data.



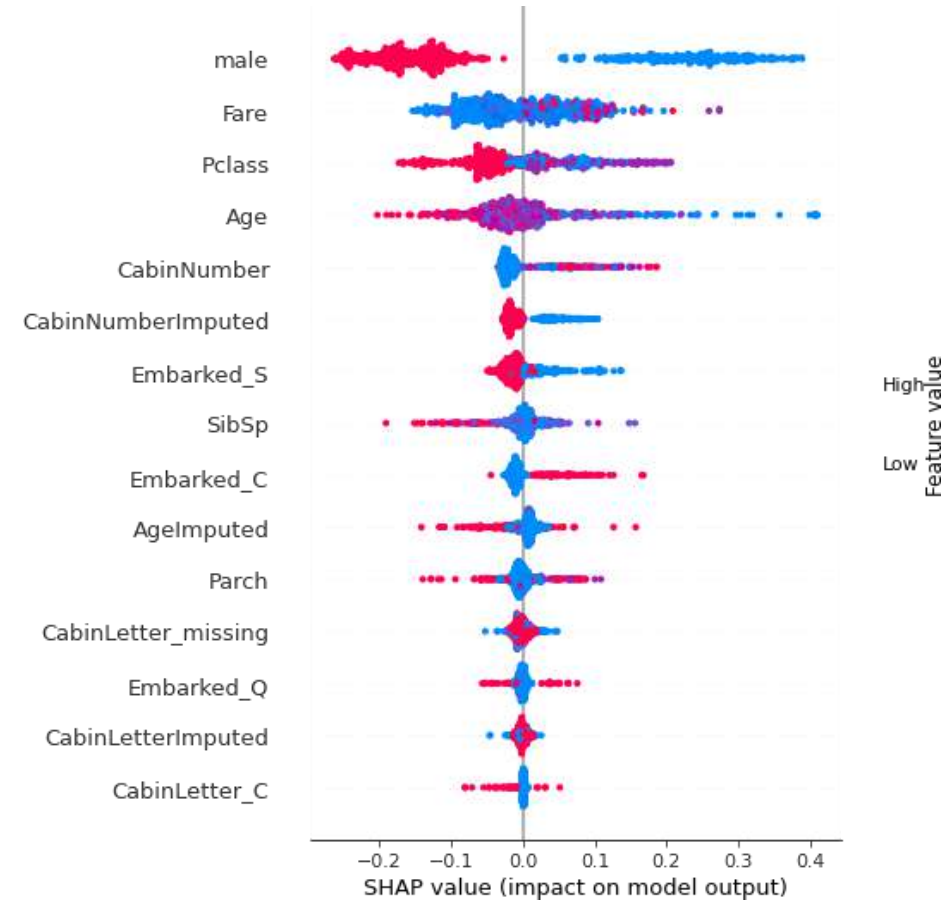
SHAP- SHapley Additive exPlanations

- SHAP is a Python library (and framework) that applies Shapley values to explain machine learning models.
- It quantifies how each feature contributes to individual predictions. Supports tree-based models (like XGBoost, LightGBM), linear models, and even deep learning models.
- Visualizes feature contributions with plots like: Force plots, Summary plots, Waterfall Plots etc

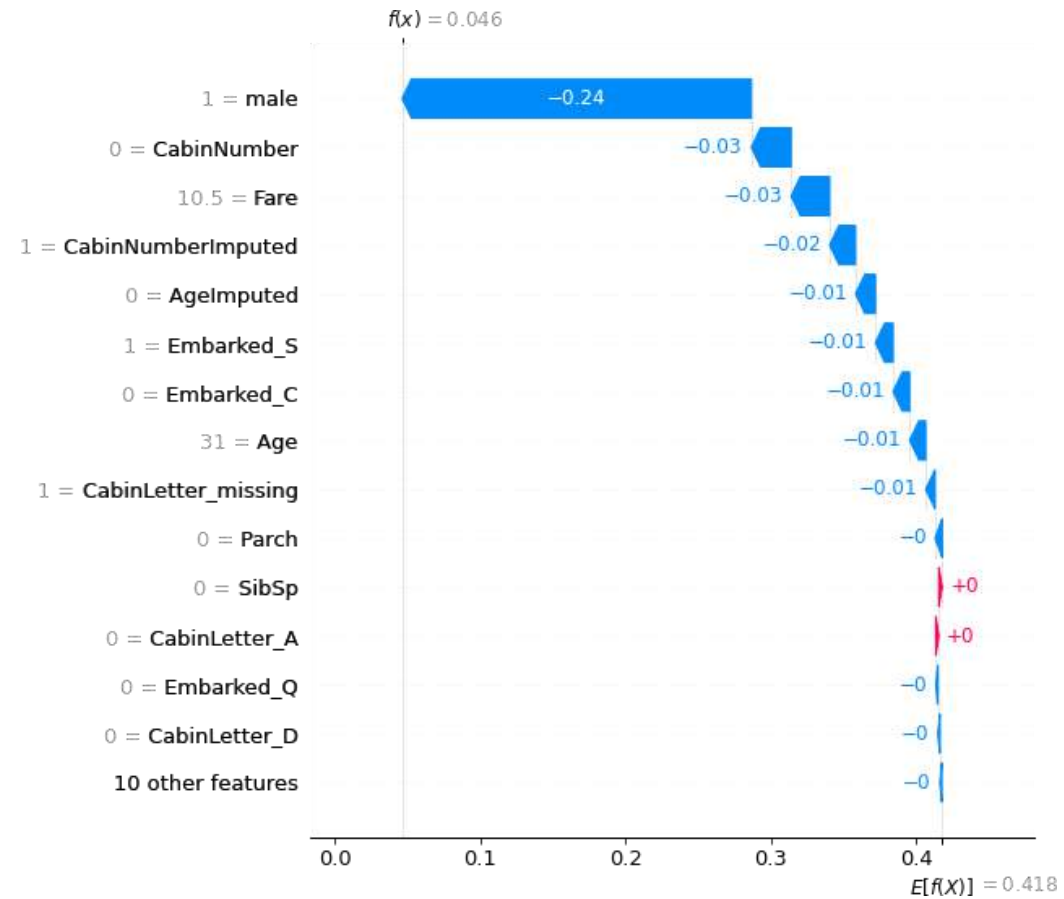
SHAP-Summary Plot



Beeswarm plot



Waterfall plot



- `display = PartialDependenceDisplay.from_estimator(`
- `model,`
- `X_test,`
- `features,`
- `kind='individual', # 'average' for PDP, 'individual' for ICE, 'both' for both`
- `subsample=50, # limit number of ICE lines`
- `grid_resolution=20,`
- `random_state=42`
- `)`
- `#plt.suptitle("PDP for 'MedInc'")`
- `plt.suptitle("ICE Plot for 'MedInc'")`
- `plt.show()`

Accumulated Local Effects(ALE)

- Accumulated local effects describe how features influence the prediction of a machine learning model on average.
- ALE plots are a faster and unbiased alternative to partial dependence plots (PDPs).
- If features of a machine learning model are correlated, the partial dependence plot cannot be trusted.

Accumulated Local Effects(ALE)

- `pip install pyALE`
- `from pyale import ale`

Feature Grid Binning

- The range of the feature X_j is divided into intervals (bins), typically based on quantiles.
- This ensures each bin contains a roughly equal number of samples.

Local Effects Within Bins

- For each bin ALE measures how the prediction changes **locally** when X_j changes from z_k to z_{k+1} , **keeping all other features fixed**.

For each data point in the bin, compute:

$$f(x_{X_j=z_{k+1}}^{(i)}) - f(x_{X_j=z_k}^{(i)})$$

where f is the prediction function and $x^{(i)}$ is the i^{th} observation.

Average Local Effects per Bin

- Average the differences over all data points that fall into the bin. This gives the local effect for that bin.

Accumulate Effects

- Starting from a reference point (usually zero or the first bin), accumulate the average effects over bins to get the accumulated local effect.

This forms the ALE function:

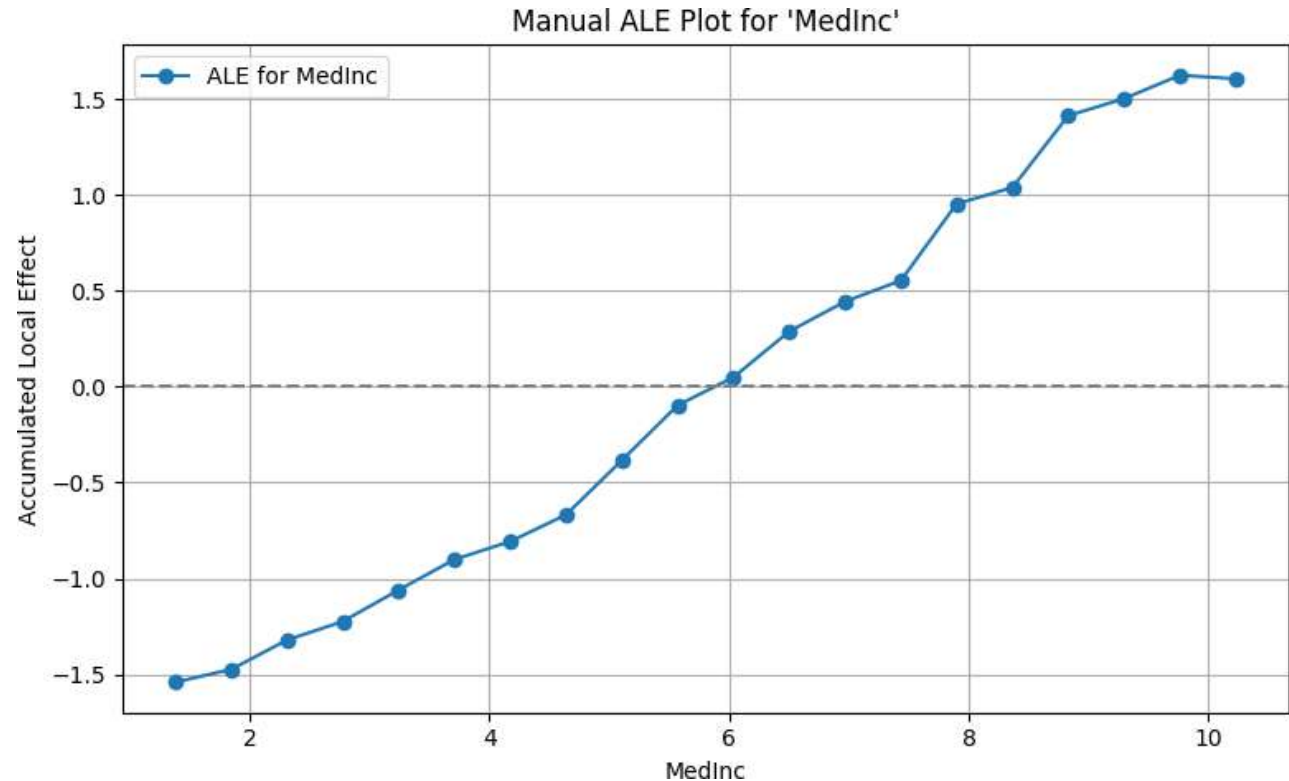
$$ALE_j(z_k) = \sum_{l=1}^{k-1} \text{Avg}_l$$

where Avg_l is the average local effect in bin l .

Centering

- The final ALE function is **centered** to have a mean of zero:

$$A\tilde{L}E_j(z_k) = ALE_j(z_k) - \frac{1}{K} \sum_{k=1}^K ALE_j(z_k)$$



Leave One Feature Out (LOFO) Importance

- The intuition behind LOFO Importance: If dropping a feature makes the predictive performance worse, then it was an important feature. If dropping the feature keeps performance unchanged, it was not important.
- When you use LOFO Importance as information for feature selection, beware of the interpretation: LOFO only indicates how the model performance reacts to *individually* removing features.
- LOFO doesn't give us the information on how the performance changes when removing 2 or more features at once.

Surrogate model

- A global surrogate model is an interpretable model that is trained to approximate the predictions of a black box model.
- Their purpose is to explain the behavior of the original model by providing a transparent proxy that mimics its output.

Surrogate model

Perform the following steps to obtain a Surrogate model

- Select a dataset .This can be the same dataset that was used for training the black box model or a new dataset from the same distribution. You could even select a subset of the data or a grid of points, depending on your application.
- For the selected dataset get the predictions of the black box model.
- Select an interpretable model type (linear model, decision tree, ...).
- Train the interpretable model on the dataset and its predictions.
- You now have a surrogate model.
- Measure how well the surrogate model replicates the predictions of the black box model.
- Interpret the surrogate model.

Apply interpretability on Regression

- Linear regression is a statistical method that models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a linear equation to the observed data.
- A linear regression model predicts the target as a weighted sum of the feature inputs.
- The linearity of the learned relationship makes the interpretation easy.

Linear models can be used to model the dependence of a regression target y on features $\mathbf{x}$. The learned relationships can be written for a single instance i as follows:

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + \epsilon$$

The predicted outcome of an instance is a weighted sum of its p features. The betas $\beta_j, j \in 1, \dots, p$ represent the learned feature weights or coefficients. The first weight in the sum, β_0 , is called the intercept and is not multiplied with a feature. The epsilon ϵ is the error we still make, i.e. the difference between the prediction and the actual outcome.¹ These errors are assumed to follow a Gaussian distribution, which means that we make errors in both negative and positive directions and make many small errors and few large errors.

To find the best coefficient, we typically minimize the squared differences between the actual and the estimated outcomes:

$$\hat{\beta} = \arg \min_{\beta_0, \dots, \beta_p} \sum_{i=1}^n \left(y^{(i)} - \left(\beta_0 + \sum_{j=1}^p \beta_j x_j^{(i)} \right) \right)^2$$

1. Simple Linear Regression

- One independent variable
- Model:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

2. Multiple Linear Regression

- More than one independent variable
- Model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon$$

y : Target variable (what we want to predict)

x_i : Independent variables (inputs/features)

β_0 : Intercept (value of y when all $x_i = 0$)

β_i : Coefficients (slope, showing the effect of each feature)

ε : Error term (difference between predicted and actual values)

- The interpretation of a weight in the linear regression model depends on the type of the corresponding feature.
 - **Numerical feature:** An increase of feature x_j by one unit increases the prediction for y by β_j units when all other feature values remain fixed.
 - **Categorical feature:** Changing feature x_j from the reference category to the other category increases the prediction for y by β_j when all other features remain fixed.
-

Example:

Numerical Feature Interpretation :

An increase of HouseSize by one square meter increases the prediction for Price by 200 units (e.g., dollars) when all other feature values remain fixed.

Categorical Feature Interpretation :

Changing feature [feature name] from the reference category to [target category] increases the prediction for y by [coefficient value] units when all other features remain fixed.

Example: Changing feature FloorType from the reference category Carpet to Parquet increases the prediction for Price by 5000 units when all other features remain fixed.

Interpreting linear models by R-squared measurement

- R-squared tells you how much of the total variance of your target outcome is explained by the model.
- The higher R-squared, the better your model explains the data.

$$R^2 = 1 - \frac{SSE}{SST}$$

SSE is the squared sum of the error terms:

$$SSE = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

SST is the squared sum of the data variance:

$$SST = \sum_{i=1}^n (y^{(i)} - \bar{y})^2$$

SST- Total Sum of Squares

SST measures the **total variability** in the target variable y .

$$\text{SST} = \sum_{i=1}^n (y_i - \bar{y})^2$$

- y_i : actual values
- $\bar{y}$: mean of actual values
- SST tells us how much the **actual values deviate from the mean**.
- It represents the **total variation** to be explained by the model.

SSE (Sum of Squared Errors)

SSE measures the **unexplained variability** — the difference between the actual and predicted values.

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- $\hat{y}_i$: predicted values from the model
- SSE tells us how far the model's predictions are from the actual values.
- Lower SSE means **better fit**.

R^2 Value	Interpretation
0.0	The model explains none of the variability :predictions are no better than guessing the mean.
0.5	The model explains 50% of the variance .Moderate predictive power.
0.8	The model explains 80% of the variance .Good model fit.
1.0	The model explains 100% of the variance .Perfect fit (usually overfitting unless it's very simple data).
< 0	The model performs worse than just predicting the mean — typically indicates a very poor or misspecified model.

Feature Importance-t statistics

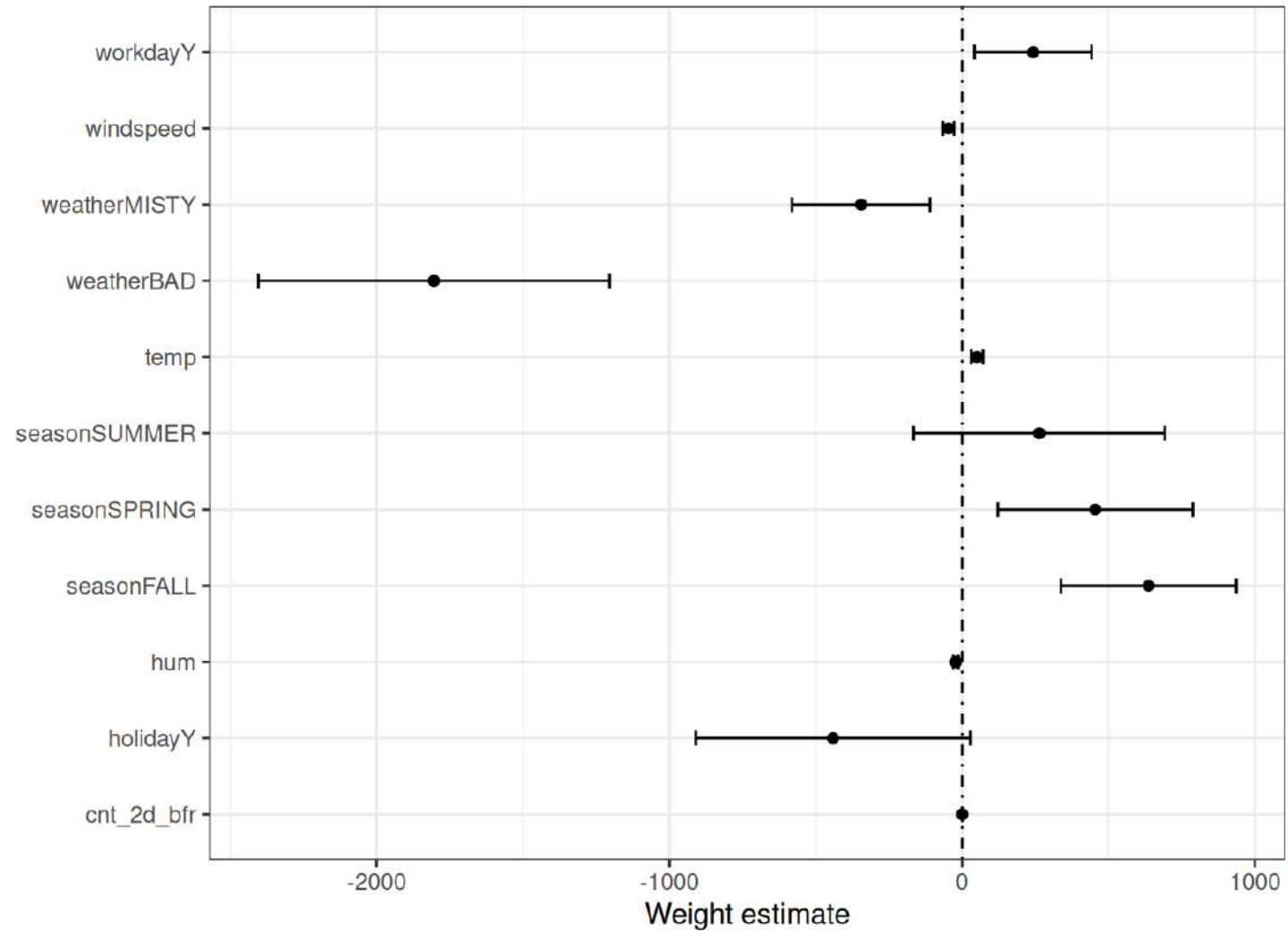
The importance of a feature in a linear regression model can be measured by the absolute value of its t-statistic. The t-statistic is the estimated weight scaled with its standard error.

$$t_{\hat{\beta}_j} = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

t statistics

- A high absolute t-value :the estimated coefficient is much larger than its standard error, meaning the feature is strong relative to noise.
- A low t-value $\rightarrow$ the feature's effect may be due to random variation, not a real relationship.
- It tests the null hypothesis that the coefficient is zero (i.e., the feature has no effect).The farther the t-value is from 0, the more likely the feature has a statistically significant impact.

Weight plot



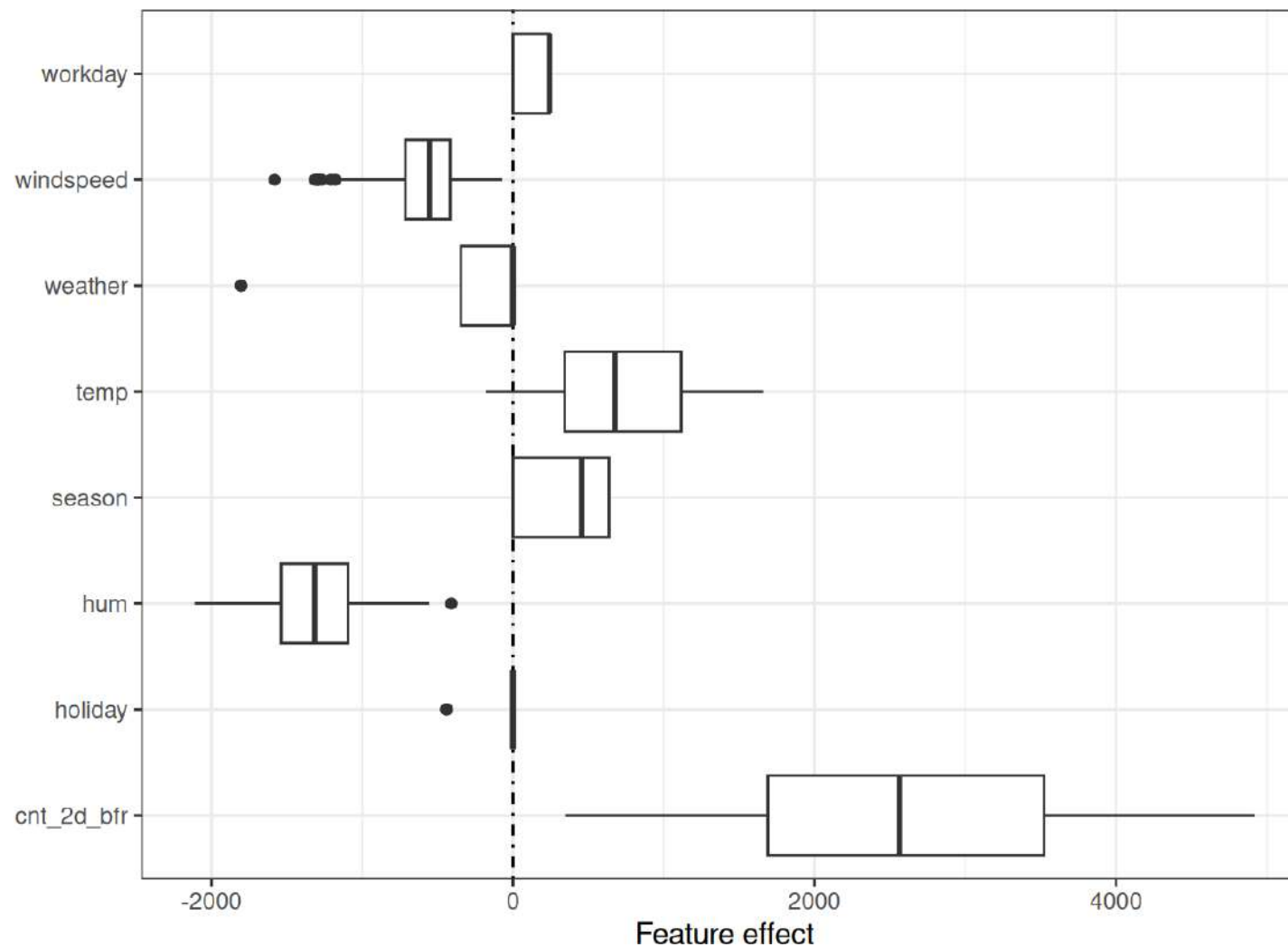
Effect Plot

- The weights of the linear regression model can be more meaningfully analyzed when they are multiplied by the actual feature values.
- The weights depend on the scale of the features and will be different if you have a feature that measures, e.g., a person's height and you switch from meter to centimeter.
- The weight will change, but the actual effects in your data will not.
- It's also important to know the distribution of your feature in the data because if you have a very low variance, it means that almost all instances have similar contributions from this feature.
- The effect plot can help you understand how much the combination of weight and feature contributes to the predictions in your data.

Effect Plot

The effects can be visualized with [boxplots](#)

$$\text{effect}_j^{(i)} = w_j x_j^{(i)}$$



Apply interpretability on Logistic regression

- Logistic regression, like linear regression, is a type of linear model that examines the relationship between predictor variables (independent variables) and an output variable (the response, target or dependent variable).
- The key difference is that **linear regression** is used when the output is a **continuous value** , for example, predicting someone's credit score. **Logistic regression** is used when the outcome is **categorical**, such as whether a loan is approved or not.

Logistic regression

- Instead of fitting a straight line or hyperplane, the logistic regression model uses the logistic function to squeeze the output of a linear equation between 0 and 1.
- The **logistic function** is defined as:

$$\text{logistic}(\mathbf{z}) = \frac{1}{1 + \exp(-\mathbf{z})}$$

- In the linear regression model, we have modeled the relationship between outcome and features with a linear equation:

$$\hat{y}^{(i)} = \beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}$$

For classification, we prefer probabilities between 0 and 1, so we wrap the right side of the equation into the logistic function. This forces the output to assume only values between 0 and 1.

$$\mathbb{P}(Y^{(i)} = 1) = \text{logistic}(\mathbf{x}^{(i)T} \boldsymbol{\beta}) = \frac{1}{1 + \exp(-(\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}))}$$

- The **interpretation of the weights** in logistic regression differs from the interpretation of the weights in linear regression since the outcome in logistic regression is a value between 0 and 1.
- The weights don't influence the probability linearly any longer.
- To interpret the weights, we need to reformulate the equation for the interpretation so that only the linear term is on the right side of the formula

$$\ln \left(\frac{\mathbb{P}(Y = 1)}{1 - \mathbb{P}(Y = 1)} \right) = \ln \left(\frac{\mathbb{P}(Y = 1)}{\mathbb{P}(Y = 0)} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Term in the $\ln()$ function “odds” (probability of event divided by probability of no event), and wrapped in the logarithm, it is called **log odds**.

This formula shows that the logistic regression model is a linear model for the log odds.

- How does the prediction change when one of the features is changed by 1 unit?

$$\frac{\mathbb{P}(Y = 1)}{1 - \mathbb{P}(Y = 1)} = \text{odds} = \exp(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)$$

Then we compare what happens when we increase one of the feature values by 1. But instead of looking at the difference, we look at the ratio of the two predictions:

$$\frac{\text{odds}_{x_j+1}}{\text{odds}_{x_j}} = \frac{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_j(x_j + 1) + \dots + \beta_p x_p)}{\exp(\beta_0 + \beta_1 x_1 + \dots + \beta_j x_j + \dots + \beta_p x_p)}$$

We apply the following rule:

$$\frac{\exp(a)}{\exp(b)} = \exp(a - b)$$

And we remove many terms:

$$\frac{\text{odds}_{x_j+1}}{\text{odds}_{x_j}} = \exp(\beta_j(x_j + 1) - \beta_j x_j) = \exp(\beta_j)$$

A change in a

feature by one unit changes the odds ratio (multiplicative) by a factor of $\exp(\beta_j)$.

We could also interpret it this way: A change in $x_j^{(i)}$ by one unit increases the log odds ratio by the value of the corresponding weight.

These are the interpretations for the logistic regression model with different feature types:

- Numerical feature: If you increase the value of $x_j^{(i)}$ by one unit, the estimated odds change by a factor of $\exp(\beta_j)$.
- Binary categorical feature: One of the two values of the feature is the reference category (in some languages, the one encoded in 0). Changing $x_j^{(i)}$ from the reference category to the other category changes the estimated odds by a factor of $\exp(\beta_j)$.
- Categorical feature with more than two categories: One solution to deal with multiple categories is one-hot-encoding, meaning that each category has its own column. You only need $L-1$ columns for a categorical feature with L categories, otherwise it is over-parameterized. The L -th category is then the reference category. You can use any other encoding that can be used in linear regression. The interpretation for each category then is equivalent to the interpretation of binary features.
- Intercept β_0 : When all numerical features are zero and the categorical features are at the reference category, the estimated odds are $\exp(\beta_0)$. The interpretation of the intercept weight is usually not relevant.

Variable	Odds Ratio	2.5% (Lower CI)	97.5% (Upper CI)
const	2.89	0.83	1.36
bmi	inf	1.01×10^7	6.85×10^{12}
bp	inf	2.56×10^2	4.24×10^7
age	9.25	0.0077	641.2

bmi OR = ∞ → Indicates numerical instability or perfect separation.
Confidence Interval : Very wide — from 10 million to 6 trillion.
Interpretation: There's a strong positive relationship, but the model is unstable or overfitted.
You may have too few examples, strong multicollinearity, or extreme values in BMI.

The extremely high odds ratios (inf) and very wide confidence intervals suggest:

- Severe multicollinearity between features (e.g., age and BMI might be correlated).
- Small sample size.
- Separation: The model perfectly predicts the outcome for some feature values .
- logistic regression can't estimate finite coefficients.
- Unscaled or outlier-heavy features.